FAST MATRIX FACTORIZATION FOR ONLINE RECOMMENDATION WITH IMPLICIT FEEDBACK

Report

April 3, 2025

Paul-Philip Louis, Guillaume Coutière

CONTENTS

Introduction **3**

1 Description of the algorithm **4**

2 Description of our implementation **7**
2.1 Context 7
2.2 Initialization 7
2.3 Iterations 8
2.3.1 S matrices 8
2.3.2 P and Q matrices 8

3 Results **9**

4 Strenghts and weaknesses of the implementation **10**

Conclusion **11**

Appendix **12**

INTRODUCTION

Recommender systems play a crucial role in helping users navigate the overwhelming volume of available choices in digital platforms, from e-commerce and streaming services to online learning and social networks. These systems personalize content, improving user experience and engagement while driving business value. However, as the diversity of offer grows, as well as user expectations for accuracy and relevance, so does the need for larger and more diverse datasets. As a result, handling large-scale datasets has become a fundamental challenge, requiring efficient algorithms and scalable infrastructure.

In their article *Fast Matrix Factorization for Online Recommendation with Implicit Feedback*, Xiangnan et al. propose an easily parallelizable algorithm for matrix factorization for recommendation systems, meant to adress the challenges of huge markets. Their approach takes inspiration from the element-wise Alternating Least Squares (eALS).

Given the increasing need to process large-scale datasets in distributed environments, Spark provides a suitable framework to implement the proposed matrix factorization technique : it provides efficient parallel computation, fault tolerance, and scalability, making it ideal for reproducing and extending the study's findings.

Our work consists in implementing their work in Python, with PySpark and experiment on it. We aim at :

- verifying the method's accuracy
- analyzing its performance in terms of execution time and resource utilization
- identifying weaknesses and potential improvements in our implementation

1 DESCRIPTION OF THE ALGORITHM

A classic way to develop a recommender system is to build a user-item ratings matrix $\mathbf{R} = (r_{ui})$, with u the indices of users and i the indices of items. Using only the available data, such a matrix is generally sparse. In order to estimate the full matrix with a reasonable number of parameters, we can assume this matrix is low-rank, and can be factorized into two smaller matrices, one of them ($\mathbf{P}$) representing users and the other one, $\mathbf{Q}$ representing items, as *latent feature vectors*. We will write $\mathbf{p}_u$ the latent feature vector for user u, $\mathbf{q}_i$ the latent feature vector for item i, and K the dimension of these vectors.
By learning these simpler representations from the available data, we can predict the missing values by the equation $\hat{r}_{ui} = \langle \mathbf{p}_u, \mathbf{q}_i \rangle = \mathbf{p}_u^\top \mathbf{q}_i$

One popular approach to factorize these matrices is Alternating Least Squares (ALS). The principle is to minimize a certain weighted cost :

$$J = \sum_{u=1}^{M} \sum_{i=1}^{N} (r_{ui} - \hat{r}_{ui})^2 + \lambda \left(\sum_{u=1}^{M} \|\mathbf{p}_u\|^2 + \sum_{i=1}^{N} \|\mathbf{q}_i\|^2 \right)$$

Minimizing it with respect to each latent feature vector is equivalent to solve ridge regressions. However, this implies a matrix inversion for each vector, leading to a huge complexity in $O((M+N)K^3 + MNK^2)$, where M and N denote the number of users and items, respectively.

To solve this problem, a solution is to minimize independently each component of each vector : this is the element-wise ALS (eALS).

In their paper *Fast Matrix Factorization for Online Recommendation with Implicit Feedback*, Xiangnan et al. focus on **implicit feedback** (e.g user's video viewing history, where we only know whether there has been interaction) and take inspiration from this algorithm to propose a "fast eALS" presenting two main changes:

- Introduces an idea of popularity-aware weighting
- Optimizes the existing eALS through memoization

In the context of implicit feedback, every missing entry corresponds either to a negative or an unknown feedback. In order to identify which are more likely to be negative feedbacks, the authors introduce the idea of *popularity*. The more an item is popular (ie many ratings), the more an unknown rating for this item is likely to be negative, for the user in question has probably been suggested the article, and decided not to interact with it. Therefore, the authors define the *popularity* of an item i as

$$c_i = c_0 \frac{f_i^\alpha}{\sum_{j=1}^{N} f_j^\alpha}$$

with c_0 the overall weight of missing data, f_i the frequency of the item i in the data, and $\alpha = 0.5$ usually (empirically the best).

The loss to minimize is rewritten, with weights (w_{ui}) :

$$L = \sum_{(u,i)\in\mathcal{R}} w_{ui}(r_{ui} - \hat{r}_{ui})^2 + \sum_{u=1}^{M} \sum_{i\notin\mathcal{R}_u} c_i \hat{r}_{ui}^2 + \lambda \left(\sum_{u=1}^{M} \|\mathbf{p}_u\|^2 + \sum_{i=1}^{N} \|\mathbf{q}_i\|^2 \right)$$

.

where $\mathcal{R}$ is the set of all pairs (u, i) which there has been interaction between, and $\mathcal{R}_u$ is the set of items user u has interacted with.

From then on, we can write the following update equations for the components of $\mathbf{p}_u$ and $\mathbf{q}_i$:

$$p_{uf} = \frac{\sum_{i \in \mathcal{R}_u} [w_{ui} r_{ui} - (w_{ui} - c_i)\hat{r}^f_{ui}] q_{if} - \sum_{k \neq f} p_{uk} s^q_{kf}}{\sum_{i \in \mathcal{R}_u} (w_{ui} - c_i) q^2_{if} + s^q_{ff} + \lambda} \tag{1}$$

$$q_{if} = \frac{\sum_{u \in \mathcal{R}_i} [w_{ui} r_{ui} - (w_{ui} - c_i)\hat{r}^f_{ui}] p_{uf} - c_i \sum_{k \neq f} q_{ik} s^p_{kf}}{\sum_{u \in \mathcal{R}_i} (w_{ui} - c_i) p^2_{uf} + c_i s^p_{ff} + \lambda} \tag{2}$$

with $\mathbf{S}^q = \sum_{i=1}^{N} c_i \mathbf{q}_i \mathbf{q}_i^T$ and $\mathbf{S}^p = \mathbf{P}^T \mathbf{P}$.

This leads the authors to suggest the following Algorithm 1 :

This algorithm is *embarrassingly parallel* :

- Each component is updated independently, making it possible to share the computations across several worker nodes.
- The matrices $\mathbf{S}^q$ and $\mathbf{S}^p$ are cached for memoization, gaining even more efficiency
- After computation of $\mathbf{S}^q$, user updates require only local data ($\mathcal{R}_u$)
- After computation of $\mathbf{S}^p$, item updates need only local data ($\mathcal{R}_i$)

On the complexity level, this element-wise approach reduces time complexity from $O(K^3)$ in classic ALS to $O(K)$ per factor update. This enables handling:

- Large latent dimensions ($K > 1000$)
- Sparse datasets with millions of users/items
- Frequent online updates with low latency

Finally, the parallelism comes at *zero approximation cost* - unlike SGD-based methods, there are no race conditions or update conflicts because:

- User/item factors are updated sequentially within each entity
- Shared $\mathbf{S}$ matrices are read-only during update phases
- No stochastic sampling introduces variance

The authors also adapt this algorithm for an online setting — however, we have focused ourselves on the offline setting in our implementation.

```
Input: R, K, λ, W, item confidence vector c
Output: Latent feature matrices P and Q
Randomly initialize P and Q;
for (u, i) ∈ R do
    r̂_ui ← p_u^⊤ q_i;
    ;                                                        // O(|R|K)
end
while stopping criteria not met do
    Update user factors:;
    Compute S^q = Σ_{i=1}^{N} c_i q_i q_i^⊤;
    ;                                                        // O(NK^2)
    for each user u do
        for each factor f do
            for each item i ∈ R_u do
                r̂^f_ui ← r̂_ui − p_uf q_if;
            end
            Update p_uf using Equation (1);
            ;                                                // O(K + |R_u|)
            for each item i ∈ R_u do
                r̂_ui ← r̂^f_ui + p_uf q_if;
            end
        end
    end
    Update item factors:;
    Compute S^p = P^⊤ P;
    ;                                                        // O(MK^2)
    for each item i do
        for each factor f do
            for each user u ∈ R_i do
                r̂^f_ui ← r̂_ui − p_uf q_if;
            end
            Update q_if using Equation (2);
            ;                                                // O(K + |R_i|)
            for each user u ∈ R_i do
                r̂_ui ← r̂^f_ui + p_uf q_if;
            end
        end
    end
    end
    return P, Q;
```

Algorithm 1: Fast eALS Learning Algorithm

2 DESCRIPTION OF OUR IMPLEMENTATION

2.1 Context

Our approach of the implementation of this algorithm relied heavily on sparse representations of matrices, which is rather logical since the goal of the algorithm is to fill a sparse matrix. Consequently, we represented most matrices by dataframes with the schema `(Row, Column, Value)`. For clarity and organization, we did not name the columns as such but rather `item`, `user`, `p`, etc., but the general idea was to represent matrices in this format. To better understand the functioning of this representation, we provided a function `matrix_multiplication` that performs a matrix multiplication between two matrices represented as sparse matrices. Though we ended up not using this function as such, we kept it as an example of the use of such matrices.

Some matrices, however, were too dense to be efficiently kept as sparse representations, so we decided to encode and use them the usual way. This was notably the case for the S^q and S^p matrices. For the same reason, since the vectors of P and Q are supposed to be dense, we stored them as `Vectors`.

2.2 Initialization

To start with, we initialize R as a sparse matrix, and C as a dataframe with schema `(item, c)`, as it is a vector.

As mentioned, we chose to encode P and Q with only a `row` and a `value` columns, because those matrices are dense, and will get to be multiplied by themselves, in which case a sparse encoding would require a `join` operation, which is costly in such cases.

We also made the choice to join C to Q from the beginning, as one is rarely used without the other, and therefore not encoding it as such would have required constantly joining them the same way, which would have been inefficient.

```
def initialize_random_vect(R, K, C=None):
  def init_rand(v):
    """ Initializes random vectors """

  C = C if C is not None else c(R)
  P = R.select("user").distinct()\
        .withColumn("p", udf_rand(col('user')))
  Q = R.select("item").distinct()\
        .join(C, on="item")\
        .withColumn("q", udf_rand(col('item')))
  return P, Q
```

Figure 1: Initialization of P and Q, joining C to Q

Similarly, since the only elements of $\hat{\mathcal{R}}$ used in the algorithm are those in $\mathcal{R}$, we joined the dataframes to save computation afterwards. Moreover, we can note that computing $\hat{\mathcal{R}}$ with sparse representations for P and Q would have required joining Q on `(item, coordinate)`, which would have been more consuming as each `item` had all `coordinates`.

```
def compute_r_hat_vector(R, P, Q):
  def scalar(v1, v2):
    """ Computes the scalar product of v1 and v2 """

  return R.select("user", "item", "r").join(P, on="user").join(Q, on="item")\
    .withColumn("r_hat", scalar_udf(col("q"), col("p")))\
    .select("user", "item", "r_hat", "r")
```

Figure 2: Initialization of $\hat{\mathcal{R}}$

2.3 Iterations

2.3.1 • S matrices

As mentioned earlier, S^q and S^p are dense matrices, and mostly used in this context. Consequently, the most efficient choice was to encode them as `numpy` matrices, though their computations were still parallelized with respect to the rows of their associated matrices (`item` for S^q and `user` for S^p), which is likely the higher dimension.

```
def compute_Sq(Q, K):
  def mult_vect(u:np.ndarray, c:float):
    """ Computes u @ u.T"""

    def sum_matrices(matrices):
    """ Sums dense matrices """

  return Q.withColumn("sq", mult_udf(col("q"), col("c")))\
          .agg(sum_matrices_udf(collect_list("sq")).alias("Sq"))
```

Figure 3: Computation of the S matrices

2.3.2 • P and Q matrices

In order to update P and Q, the first step is to compute the current values of $\hat{r}^f_{ui}$ from P and Q, which requires joining P, Q, and $\hat{\mathcal{R}}$. Since this operation is also necessary in the computation of the new values of P and Q, we keep a base dataframe `R_temp`, which enables us to compute the new values of P and Q on the one hand, and to update the values of $\hat{\mathcal{R}}$ on the other hand.

The important point to note here is that since the observations are independent, we can parallelize over `u` and `i`, simply by switching the loops.

```
    for f in tqdm(range(K)):
        # Parallelization over observation, dimensions updated one by one
        Q, Rs = compute_iteration(Rs, Q, P, Sp, f, K, spark)
```

Figure 4: Computation of the P and Q matrices

3 RESULTS

To evaluate the performance of our implementation, we measured the time required to run the algorithm on datasets of varying sizes (subsets of the Shoe dataset : https://snap.stanford.edu/data/amazon/Shoes.txt.gz). As illustrated in Figure 5, the execution times exhibit a relatively stable pattern across different dataset sizes, with minor fluctuations due to inherent variations in computational load distribution.

For smaller datasets, the runtime remains consistent, hovering around 70 to 75 seconds for most cases. Notably, increasing the dataset size from 1,000 to 10,000 samples does not introduce significant overhead, with execution times fluctuating between approximately 72 and 75 seconds. This suggests that our approach efficiently handles moderate-scale data without major computational bottlenecks.

However, as the dataset size grows beyond 10,000 observations, we observe a more noticeable increase in runtime. For instance, at 20,000 samples, the algorithm runs in 82.46 seconds, while at 50,000 samples, it reaches 98.73 seconds. Finally, for the largest dataset tested (100,000 observations), the execution time extends to 107.71 seconds, indicating a gradual but controlled rise in computational cost as dataset size scales up.

These results highlight the efficiency of our sparse matrix representation and parallelized computations, as the implementation maintains reasonable execution times even for large datasets. The observed increase in runtime is expected given the iterative nature of the algorithm, as each iteration involves matrix updates and joins, which become more expensive with larger datasets. Nevertheless, the relatively smooth scaling suggests that our design choices—particularly the strategic use of sparse and dense representations—successfully mitigate unnecessary computational overhead, allowing the implementation to handle increasing data sizes with manageable performance trade-offs.

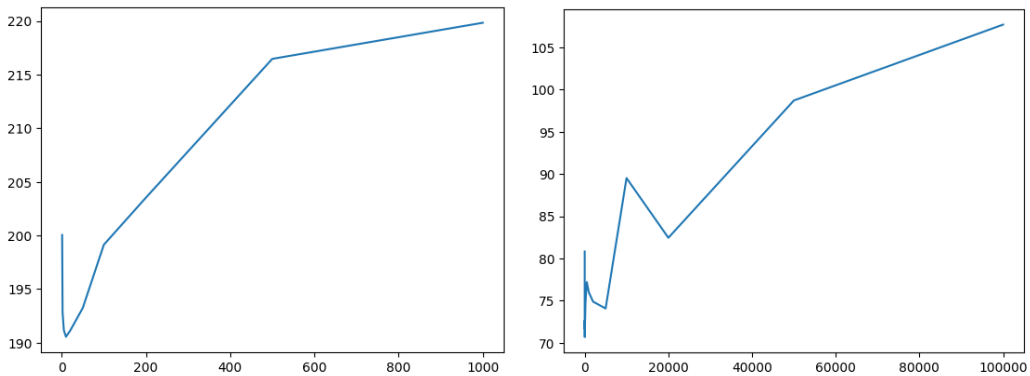

Figure 5: Time achieved to run the algorithm with higher (left) and lower (right) iterations and hidden dimensions, in function of the size of the dataset.

4
STRENGHTS AND WEAKNESSES OF THE IMPLEMENTATION

One of the main weaknesses of this implementation comes from handling recursion in Spark schemas, due to the iterative nature of the algorithm. Since each iteration required joining updated versions of previously computed dataframes, the schema grew increasingly complex, leading to deeply nested structures that Spark struggled to manage efficiently.

To address this issue, we introduced regular checkpointing to break the recursive schema and simplify the lineage of computations. While this approach successfully mitigated the problem, it introduced a significant trade-off: checkpointing was inherently time-consuming, as it required materializing intermediate results and reloading them before proceeding with the next iteration. This added computational overhead, but it was necessary to ensure the algorithm remained manageable and performant within the Spark environment.

Another weakness of our implementation is the use of many user-defined functions, that might slow the process compared to native pyspark functions. Finally, the use of many joins implies a lot of shuffling that is computationally expensive.

However, one of the key advantages of our approach lies in its efficiency in handling sparse data, as the algorithm is designed to operate directly on sparse representations without requiring costly conversions. By structuring matrices as dataframes with minimal storage overhead, we significantly reduced memory consumption and improved scalability, making it well-suited for large datasets. Additionally, the iterative framework allowed for parallelization at multiple levels, whether in computing updates for P and Q or in aggregating values for S^p and S^q, ensuring optimal resource utilization.

Another strength of the algorithm is its adaptability; by carefully choosing when to maintain sparsity and when to leverage dense structures, we were able to strike a balance between computational efficiency and ease of manipulation, minimizing unnecessary operations such as costly joins. Finally, by pre-joining frequently used data structures, we reduced redundant computations, further enhancing the overall performance of the algorithm.

CONCLUSION

In this project, we successfully implemented the Fast eALS matrix factorization algorithm in Python using PySpark, aimed at efficiently handling large-scale recommendation datasets. Our approach took advantage of Spark's distributed computing capabilities, ensuring the algorithm's scalability and parallelization. By representing the matrices in sparse formats where applicable, we minimized memory consumption and optimized computational efficiency. This design choice allowed the algorithm to perform well even with large datasets, a key feature of the Fast eALS method, which is specifically designed for large-scale and sparse recommendation problems.

Throughout the implementation, we closely followed the principles outlined in the paper by Xiangnan et al., adapting their algorithm for offline use in a distributed setting. One of the key features of the Fast eALS algorithm is its parallelization, which we leveraged extensively. The algorithm's element-wise updates for the user and item factors enabled efficient parallel processing of large matrices. By separating the computations for user and item factors, and using memoization for the S^q and S^p matrices, we reduced redundant computations, making the algorithm more efficient overall. This approach allowed us to achieve strong performance, with execution times remaining manageable even as dataset sizes grew.

However, one challenge we encountered was the iterative and recursive nature of the algorithm's data processing in Spark. As each iteration involved updating large matrices and performing multiple joins, the schema grew increasingly complex, which led to performance bottlenecks. To mitigate this, we introduced checkpointing, which helped break up the recursive lineage and simplify the computational graph. While this approach was effective in managing the computational complexity, it came at the cost of additional overhead due to the materialization of intermediate results. This trade-off between performance and stability is something that could be further optimized in future implementations.

Another significant aspect of our implementation was the careful choice of when to use sparse versus dense data structures. By keeping matrices like S^q and S^p as dense structures, we avoided the costly operations that would have been required if we had tried to maintain them as sparse matrices. Similarly, pre-joining frequently used data structures, like the P and Q matrices, reduced redundant computations and improved the overall performance. This approach allowed us to strike a balance between computational efficiency and ease of manipulation, enabling the algorithm to scale effectively with larger datasets.

Looking ahead, there are several potential improvements and extensions to this implementation. One area for future work could involve refining the management of sparse data representations, especially for matrices like S^q and S^p, by experimenting with more efficient storage formats or further optimizing joins. Additionally, while our implementation focused on the offline setting, adapting the algorithm for online recommendations could offer significant improvements for real-time recommendation systems, aligning with the original paper's goals.

Finally, while the performance of the algorithm remained stable across varying dataset sizes, future testing on even larger datasets, as well as benchmarking against other matrix factorization techniques, would provide deeper insights into the algorithm's scalability and comparative advantages.

In conclusion, this project successfully demonstrated the implementation of the Fast eALS algorithm in a distributed computing framework, validating its potential for handling large-scale recommendation problems. Through careful design choices and optimizations, we were able to maintain efficiency while scaling the algorithm. While there are areas for further improvement, the implementation provides a solid foundation for both offline and online recommendation systems in big data environments.

APPENDIX

```
import pyspark
from pyspark import SparkContext
from pyspark.sql import SparkSession
from pyspark.sql.functions import regexp_extract, col, lit, pow as spark_pow, sum\
as spark_sum, when, rand, randn, collect_list
from pyspark.sql import Row, Column
import numpy as np
from tqdm import tqdm
from pyspark import StorageLevel
from pyspark.ml.linalg import Vectors, VectorUDT, DenseMatrix, MatrixUDT
from pyspark.sql.functions import udf
from pyspark.sql.types import FloatType

import time
import pandas as pd

import requests
import os

def download_file(url):
    local_filename = url.split('/')[-1]
    if os.path.exists(local_filename):
        print(f"{local_filename} already exists")
        return local_filename
    # NOTE the stream=True parameter below
    with requests.get(url, stream=True) as r:
        r.raise_for_status()
        with open(local_filename, 'wb') as f:
            for chunk in r.iter_content(chunk_size=8192):
                # If you have chunk encoded response uncomment if
                # and set chunk_size parameter to None.
                #if chunk:
                f.write(chunk)
    return local_filename

# Extract dataframe from file
def get_df(file, limit=None, spark=None):
  df = spark.read.text(file, lineSep="\n\n")

  # Define regular expressions for extracting fields
  product_pattern = r"product/productId:\s*(\S+)"
  user_pattern = r"review/userId:\s*(\S+)"
  score_pattern = r"review/score:\s*(\S+)"

  # Extract fields
  df = df.select(
      regexp_extract("value", product_pattern, 1).alias("productId"),
      regexp_extract("value", user_pattern, 1).alias("userId"),
```

```
        regexp_extract("value", score_pattern, 1).alias("score")
    )
    return df.limit(limit) if limit else df

# Encoding the sparse matrix as a dataframe and computing basic operations
(multiplication and transposition)
def encode_matrix(df, rows, cols, values, new_row="row", new_col="col", new_val="value"):
    """
    Each row of the new df represents a matrix entry with row index, column index,
    and value.
    """
    return df.withColumnRenamed(rows, new_row)\
             .withColumnRenamed(cols, new_col)\
             .withColumnRenamed(values, new_val)

def multiply_matrices(dfA, dfB):
    """
    Multiplies two matrices stored as dfs.
    """
    # Perform join on column of A and row of B
    result = (dfA.alias("A")
              .join(dfB.alias("B"), col("A.col") == col("B.row"))
              .select(col("A.row").alias("row"), col("B.col").alias("col"),\
              (col("A.value") * col("B.value")).alias("product")))
    result = result.groupBy("row", "col").sum("product")
    result = result.withColumnRenamed("sum(product)", "value")

    return result

def transpose(df):
    return df.withColumnRenamed("row", "col_t")\
    .withColumnRenamed("col", "row")\
    .withColumnRenamed("col_t", "col")

def c(df, c0=1, alpha=.5):
    df_c = df\
            .groupby("item")\
            .count().withColumnRenamed("count", "c")\
            .withColumn("c", spark_pow(col("c"),alpha))
    sum_values = df_c.select(spark_sum("c")).collect()[0][0]
    return df_c.withColumn("c", c0*col("c")/sum_values)

def initialize_random_vect(R, K, C=None):
    def init_rand(v):
        return Vectors.dense(np.random.randn(K))

    udf_rand = udf(init_rand, VectorUDT())
    C = C if C is not None else c(R)
    P = R.select("user").distinct()\
          .withColumn("p", udf_rand(col('user')))
    Q = R.select("item").distinct()\
          .join(C, on="item")\
```

```
        .withColumn("q", udf_rand(col('item')))
  return P, Q

def compute_r_hat_vector(R, P, Q):
  def scalar(v1, v2):
    return float(np.sum(v1*v2))

  scalar_udf = udf(scalar, FloatType())

  return R.select("user", "item", "r").join(P, on="user").join(Q, on="item")\
    .withColumn("r_hat", scalar_udf(col("q"), col("p")))\
    .select("user", "item", "r_hat", "r")

def compute_Sq(Q, K):
  def mult_vect(u:np.ndarray, c:float):
    u = u.reshape(-1, 1)
    return DenseMatrix(K, K, (c * u @ u.T).flatten())

    mult_udf = udf(mult_vect, MatrixUDT())

    def sum_matrices(matrices):
      if not matrices:
          return None

      first_matrix = matrices[0]
      sum_matrix_array = first_matrix.toArray()

      for matrix in matrices[1:]:
          sum_matrix_array += matrix.toArray()
      return DenseMatrix(first_matrix.numRows, first_matrix.numCols,\
      sum_matrix_array.flatten(), isTransposed=False)

  sum_matrices_udf = udf(sum_matrices, MatrixUDT())

  return Q.withColumn("sq", mult_udf(col("q"), col("c")))\
          .agg(sum_matrices_udf(collect_list("sq")).alias("Sq"))

def compute_iteration_P(Rs, Q, P, S, k, K, spark):
  def comp_r_f(r_hat, p, q):
    return float(r_hat - p[k]*q[k])

  r_f_udf = udf(comp_r_f, FloatType())

  def recomp_r_hat(r_f, p, q):
    return float(r_f + p[k]*q[k])

  r_hat_udf = udf(recomp_r_hat, FloatType())

  def numer(r, c, rf, q):
    return float((1*float(r) - (1-c)*rf)*q[k])

  numer_udf = udf(numer, FloatType())
```

```
  def denom(c, q):
    return float((1-c)*pow(q[k], 2))

  denom_udf = udf(denom, FloatType())

  S_bc = spark.sparkContext.broadcast(S)

  def compute_sum_sq(p):
    return float(np.sum([p[i]*S_bc.value[i, k] for i in range(K) if i != k]))

  sum_sq_udf = udf(compute_sum_sq, FloatType())

  def update_P(p, p_new):
    p_array = p.toArray()
    p_array[k] = p_new
    return Vectors.dense(p_array)

  update_P_udf = udf(update_P, VectorUDT())


  R_temp = P.join(Rs, on="user")\
           .join(Q, on="item")\
           .withColumn("r_f", r_f_udf(col("r_hat"), col("p"), col("q"))).persist()

  new_P = R_temp.withColumns({
              "numerator": numer_udf(col("r"), col("c"), col("r_f"), col("q")),
              "denominator": denom_udf(col("c"), col("q"))
              })\
           .select("user", "item", "numerator", "denominator", "p")\
           .groupby("user")\
           .agg({"numerator": "sum", "denominator": "sum", "p": "first"})\
           .withColumnRenamed("sum(numerator)", "numerator")\
           .withColumnRenamed("sum(denominator)", "denominator")\
           .withColumnRenamed("first(p)", "p")\
           .withColumn("sum_sq", sum_sq_udf(col("p")))\
           .withColumn("new_p", (col("numerator") + col("sum_sq"))\
           /(col("denominator") + S_bc.value[k, k] * lit(1)))\
           .withColumn("p", update_P_udf(col("p"), col("new_p")))\
           .select("user", "p").persist()

  new_R = R_temp.drop("p").join(new_P, on="user")\
                .withColumn("r_hat", r_hat_udf(col("r_f"), col("p"), col("q")))\
                .select("user", "item", "r_hat", "r").persist()

  R_temp.unpersist()

  return new_P, new_R

def compute_Sp(P, K):
  def mult_vect(u:np.ndarray):
    u = u.reshape(-1, 1)
```

```
    return DenseMatrix(K, K, (u @ u.T).flatten())

  mult_udf = udf(mult_vect, MatrixUDT())

def sum_matrices(matrices):
      if not matrices:
          return None

      first_matrix = matrices[0]
      sum_matrix_array = first_matrix.toArray()

      for matrix in matrices[1:]:
          sum_matrix_array += matrix.toArray()
      return DenseMatrix(first_matrix.numRows, first_matrix.numCols,\
      sum_matrix_array.flatten(), isTransposed=False)

  sum_matrices_udf = udf(sum_matrices, MatrixUDT())

  return P.withColumn("sp", mult_udf(col("p")))\
  .agg(sum_matrices_udf(collect_list("sp")).alias("Sp"))

def compute_iteration_Q(Rs, Q, P, S, k, K, spark):
  def comp_r_f(r_hat, p, q):
    return float(r_hat - p[k]*q[k])

  r_f_udf = udf(comp_r_f, FloatType())

  def recomp_r_hat(r_f, p, q):
    return float(r_f + p[k]*q[k])

  r_hat_udf = udf(recomp_r_hat, FloatType())

  def numer(r, c, rf, p):
    return float((1*float(r) - (1-c)*rf)*p[k])

  numer_udf = udf(numer, FloatType())

  def denom(c, p):
    return float((1-c)*pow(p[k], 2))

  denom_udf = udf(denom, FloatType())

  S_bc = spark.sparkContext.broadcast(S)

  def compute_sum_sp(q):
    return float(np.sum([q[i]*S_bc.value[i, k] for i in range(K) if i != k]))

  sum_sp_udf = udf(compute_sum_sp, FloatType())

  def update_Q(q, q_new):
    q_array = q.toArray()
    q_array[k] = q_new
```

```
    return Vectors.dense(q_array)

  update_Q_udf = udf(update_Q, VectorUDT())


  R_temp = Q.join(Rs, on="item")\
           .join(P, on="user")\
           .withColumn("r_f", r_f_udf(col("r_hat"), col("p"), col("q")))\
           .persist()

  new_Q = R_temp.withColumns({
                "numerator": numer_udf(col("r"), col("c"), col("r_f"), col("p")),
                "denominator": denom_udf(col("c"), col("p"))
                })\
           .select("user", "item", "c", "numerator", "denominator", "q")\
           .groupby("item")\
           .agg({"numerator": "sum", "denominator": "sum", "q": "first", "c":"first"})\
           .withColumnRenamed("sum(numerator)", "numerator")\
           .withColumnRenamed("sum(denominator)", "denominator")\
           .withColumnRenamed("first(q)", "q")\
           .withColumnRenamed("first(c)", "c")\
           .withColumn("sum_sp", sum_sp_udf(col("q")))\
           .withColumn("new_q", (col("numerator") + col("sum_sp"))\
           /(col("denominator") + S_bc.value[k, k] * lit(1)))\
           .withColumn("q", update_Q_udf(col("q"), col("new_q")))\
           .select("item", "c", "q").persist()

  new_R = R_temp.drop("q").join(new_Q, on="item")\
                 .withColumn("r_hat", r_hat_udf(col("r_f"), col("p"), col("q")))\
                 .select("user", "item", "r_hat", "r").persist()

  R_temp.unpersist()

  return new_Q, new_R

def fast_eALS_parallelized(R, K, l, C, spark, n=10, checkpoint=False):

  times = []
  t = time.time()
  times.append(("Initialization", t))
  print("Initializing P and Q.")
  P, Q = initialize_random_vect(R, K)
  P, Q = P.persist(), Q.persist()
  P.count(), Q.count()
  print("P and Q computed.")
  if checkpoint:
    P = P.checkpoint(eager=True)
    Q = Q.checkpoint(eager=True)
  print("P and Q initialized.")

  print("Computing R_hat.")
  Rs = compute_r_hat_vector(R, P, Q)
```

```
Rs = Rs.persist()
Rs.count()
print("R_hat computed.")
if checkpoint:
  Rs = Rs.checkpoint(eager=True)
print("R_hat initialized.")

t = time.time()
times.append(("End of initialization", t))

for _ in range(n):

  t = time.time()
  times.append(("Beginning of Sq computation", t))

  print(f"Iteration { + 1}/{n}")
  Sq = compute_Sq(Q, K)
  Sq = Sq.collect()[0]
  Sq = Sq["Sq"].toArray()

  t = time.time()
  times.append(("End of Sq computation", t))

  print("Updating P :")
  for f in tqdm(range(K)):

    t = time.time()
    times.append((f"Beginning of P_{f} update", t))

    P, Rs = compute_iteration_P(Rs, Q, P, Sq, f, K, spark)
    P = P.persist()
    Rs = Rs.persist()
    P.count(), Rs.count()
    if checkpoint:
      P = P.checkpoint(eager=True)
      Rs = Rs.checkpoint(eager=True)

    t = time.time()
    times.append((f"End of P_{f} update", t))

  t = time.time()
  times.append((f"Beginning of Sp computation", t))

  Sp = compute_Sp(P, K)
  Sp = Sp.collect()[0]
  Sp = Sp["Sp"].toArray()

  t = time.time()
  times.append((f"End of Sp computation", t))

  print("Updating Q :")
  for f in tqdm(range(K)):
```

```
        t = time.time()
        times.append((f"Beginning of Q_{f} update", t))

        Q, Rs = compute_iteration_Q(Rs, Q, P, Sp, f, K, spark)
        Q = Q.persist()
        Rs = Rs.persist()
        Q.count(), Rs.count()
        if checkpoint:
          Q = Q.checkpoint(eager=True)
          Rs = Rs.checkpoint(eager=True)

        t = time.time()
        times.append((f"End of Q_{f} update", t))

      if checkpoint:
        P = P.checkpoint(eager=True)
        Q = Q.checkpoint(eager=True)
        Rs = Rs.checkpoint(eager=True)

  return P, Q, Rs, times

def test_algo(file_name, limit=None, K=10, l=1, n=10, checkpoint=True):
  spark = SparkSession.builder.appName("ALS").getOrCreate()
  spark.sparkContext.setCheckpointDir("/tmp/checkpoints")

  print("Initializing R.")
  df = get_df(file_name, limit, spark)
  R = encode_matrix(df, "userId", "productId", "score", "user", "item", "r").persist()
  if checkpoint:
    R = R.checkpoint(eager=True)
  print("R initialized.")
  C = c(R)
  if checkpoint:
    C = C.checkpoint(eager=True)
  print("C initialized.")

  P, Q, Rs, times = fast_eALS_parallelized(R, K, l, C, spark, n, checkpoint)
  P = P.persist()
  Q = Q.persist()
  Rs = Rs.persist()
  P.count(), Q.count(), Rs.count()

  P = P.collect()
  Q = Q.collect()
  Rs = Rs.collect()

  spark.stop()
  return P, Q, Rs, times

total_times = {}
times_results = {}
url_shoes = "https://snap.stanford.edu/data/amazon/Shoes.txt.gz"
```

```
shoes_file = download_file(url_shoes)

for lim in [10, 20, 30]:
  t0 = time.time()
  p, q, rs, times = test_algo(shoes_file, limit=lim, checkpoint=False, n=2, K=1)
  t1 = time.time()
  total_times[lim] = t1 - t0
  print(f"Total time: {t1 - t0}")
  times_results[lim] = times
  print(times)
```